

Índices Avanzados

SEMANA 05

Percy Lovon
plovon@utec.edu.pe

1.



Métodos de Organización de Archivos

Hash File

File Structures: An object-oriented approach with C++, Michael J. Folk. Addison Wesley, 3rd Edition, 1998.

Hash File

- Utiliza una función hash para asignar direcciones a los registros.
- Permite acceso rápido y directo a los registros.
- Es especialmente eficiente en búsquedas de igualdad basadas en una clave.
- Puede sufrir colisiones (múltiples registros asignados a la misma dirección).
- Se emplean técnicas de resolución de colisiones como encadenamiento o direccionamiento abierto.

Fundamentals of Database Systems, Elmasri & Navathe (Sixth Edition), Chapter 18.

Hash-based Index

- **Componentes**

- **Hash Function**
- **Hash Index**
- **Bucket/Página**

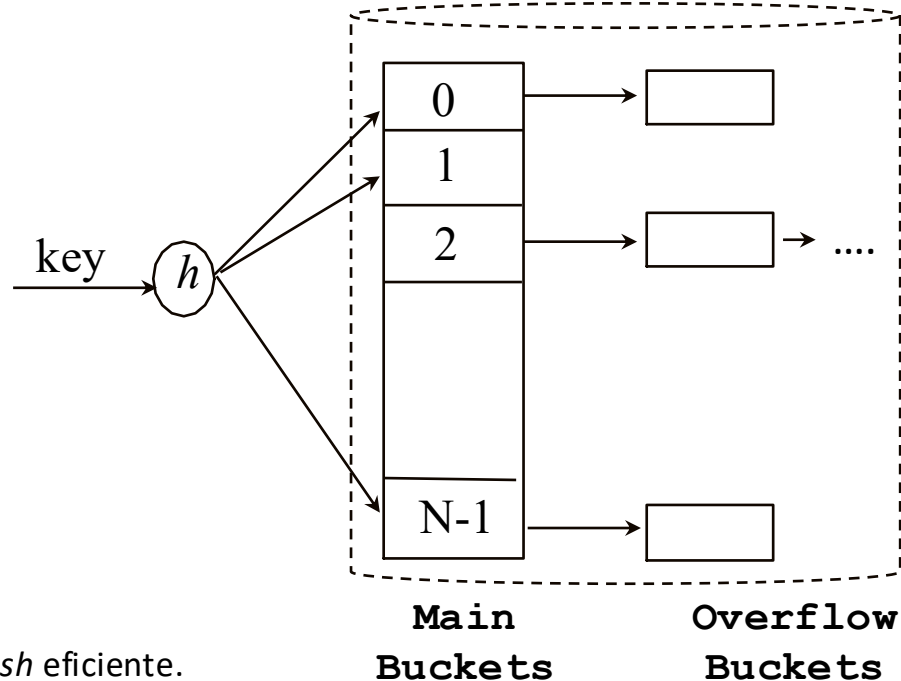
Hash Function:

$$\text{index}(\text{key}) = h(\text{key}) \bmod M.$$

key: clave de búsqueda.

M: número de bloques disponibles.

Generalmente se usa un método de *hash* eficiente.

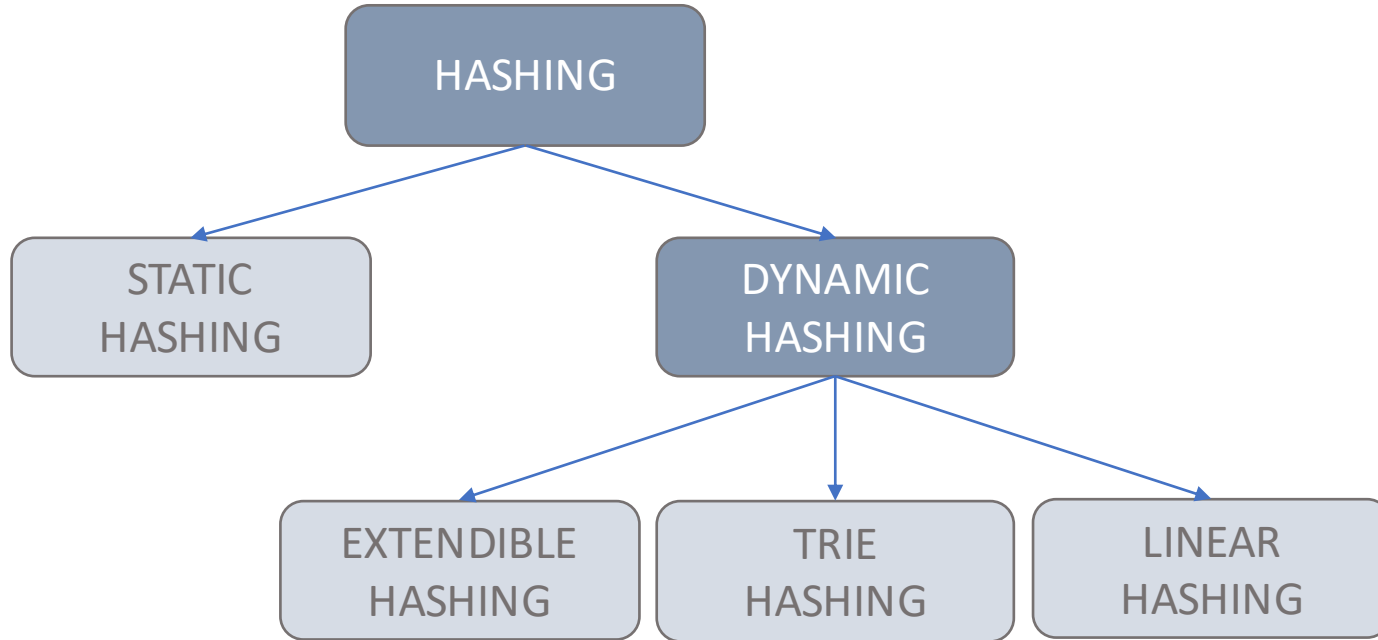


Hash-based Index

- **Componentes**

- **Hash Function:** $h(key)$ es una función de mapeo que asocia todo el conjunto de claves de búsqueda a la dirección real del bucket en memoria secundaria. La función hash debe seleccionar de manera que los registros sean esparcidos de manera uniforme. Puede darse que los registros con diferentes valores de clave de búsqueda **se asignen al mismo grupo**; por lo tanto, se debe buscar secuencialmente todo el bucket para localizar un registro.
- **Hash Index:** es el espacio de valores de la función hash y sus direcciones a los buckets de datos.
- **Bucket:** los datos son organizados en buckets. Un bucket generalmente corresponde a una **página/bloque** en donde se almacena uno o mas registros.

Hash Indexes



* Dynamic Hashing

Static Hashing

- **Definición**

- La función de hash asigna cada clave a un bucket en un **rango fijo** de direcciones.
- Cuando los buckets se llenan, se crean **áreas de overflow** (encadenamiento).

Datos.dat

0		
1		
2		
3		
4		
5		

Main
Buckets

Overflow
Buckets

Factor de bloque:
4 registros por bloque

Static Hashing

- **Ejemplo**

- **Insertar los siguientes datos:**

Keys

2,3,5

7,11,17

18,19,23

28,29,31

36,37,40

41,46,53

60,61

$$\text{index}(\text{key}) = \text{key} \bmod 6$$

Datos.dat

0		
1		
2		
3		
4		
5		

Main
Buckets

Overflow
Buckets

Factor de bloque:
4 registros por bloque

Static Hashing

- Ejemplo

- Insertar los siguientes datos:

Keys

2,3,5

7,11,17

18,19,23

28,29,31

36,37,40

41,46,53

60,61

$$\text{index}(\text{key}) = \text{key} \bmod 6$$

Datos.dat

0	18, 36, 60	
1	7,19,31,37	61
2	2	
3	3	
4	28,40,46	
5	5,11,17,23	29,53

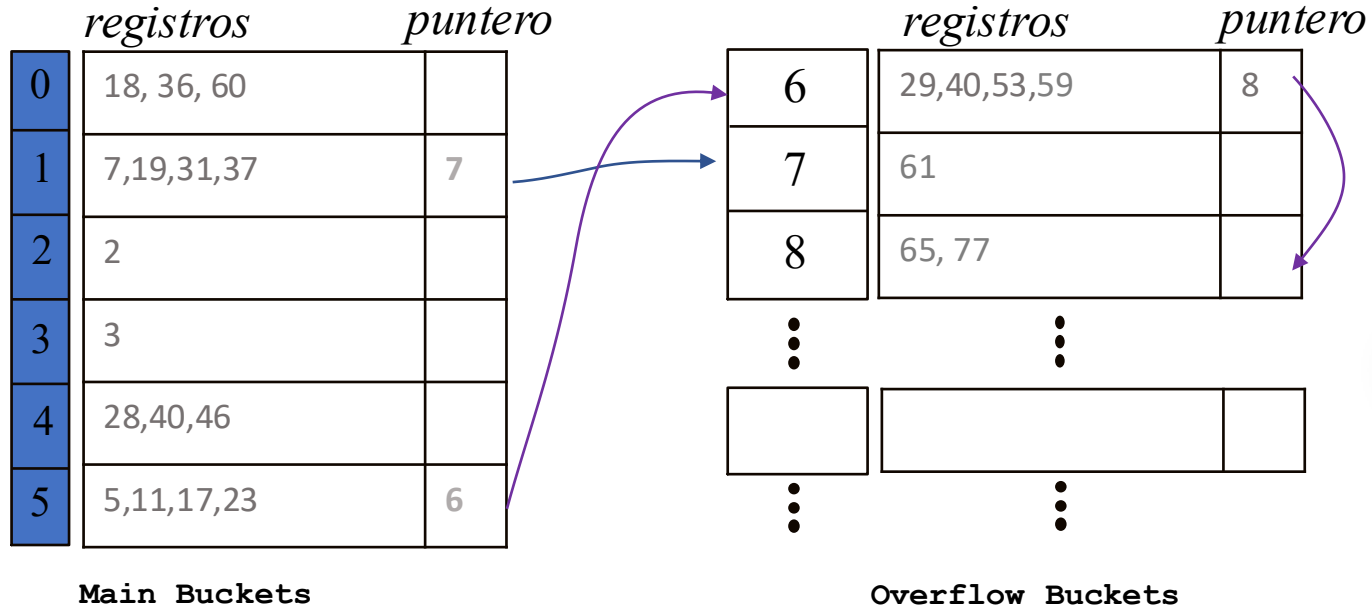
Main
Buckets

Overflow
Buckets

Factor de bloque:
4 registros por bloque

Static Hashing

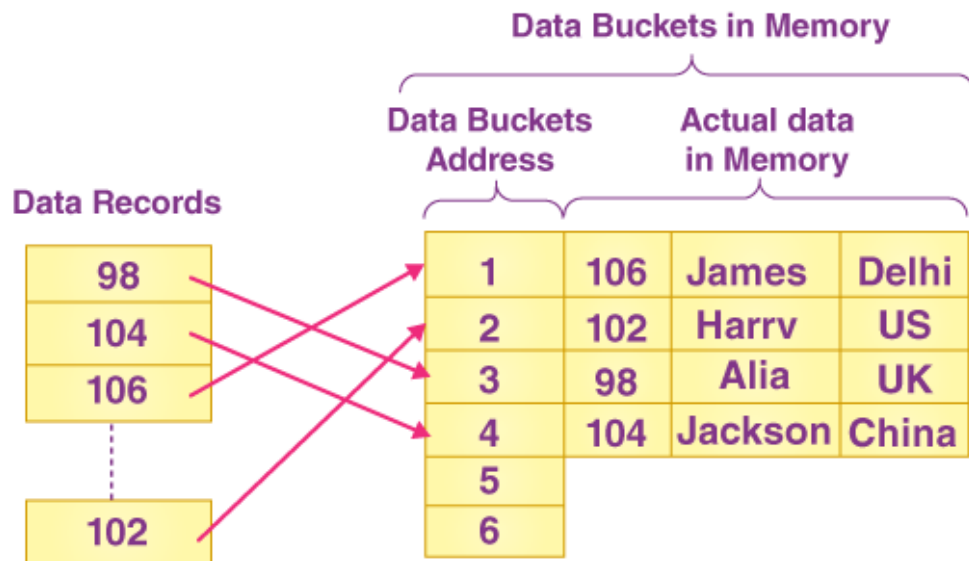
- **Manejo del overflow:** desbordamiento encadenado. SGDB reserva espacio.



Static Hashing

- **Inserción:** se usa la función hash para asignar el registro al bucket correspondiente.
 - Si no hay espacio en el bucket colocar en el desbordamiento de dicho bucket.
- **Eliminación:** localizar la pagina del registro usando la función hash.
 - Eliminar el registro del main bucket o del overflow. En este último caso, traer un elemento del overflow bucket hacia el main bucket.
- **Búsqueda:** acceso directo usando la función hash y luego recorrido lineal en los buckets desbordados.

Static Hashing



Static Hashing

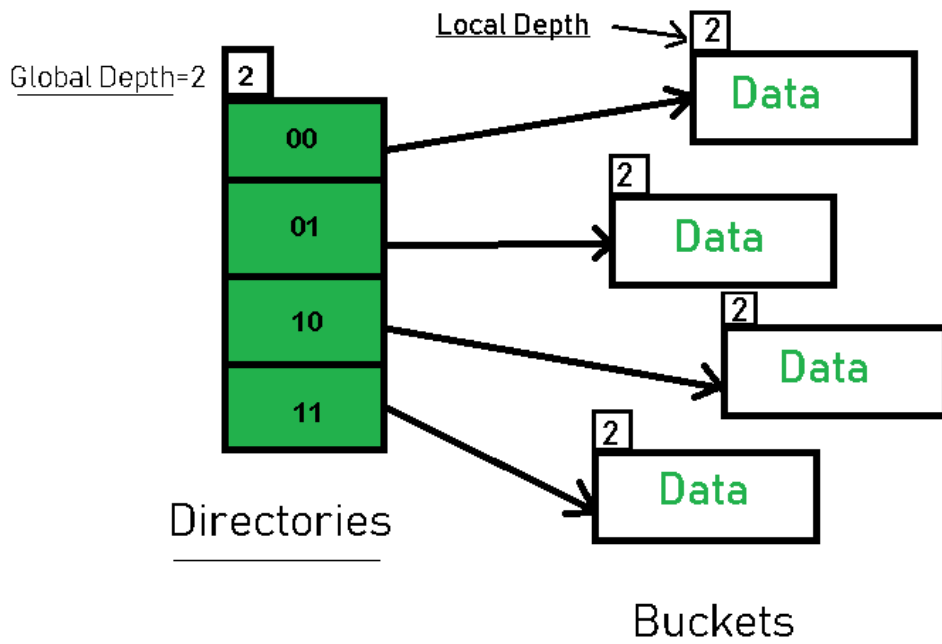


- **Deficiencias:**

- En el hash estático, la función hash mapea valores de la clave de búsqueda a un **conjunto fijo** de direcciones de buckets. Pero las bases de datos crecen o disminuyen con el tiempo.
- Posible solución: reorganización periódica del archivo con una nueva función hash.
- **Mejor solución:** permitir la modificación del número de buckets dinámicamente.

Extendible Hashing

- Introducido por Ronald Fagin et. al.(1979), *“Extendible hashing - a fast access method for dynamic files”*
- Es un tipo de hash dinámico para gestionar base de datos que crecen y reducen su tamaño en el tiempo (transaccionales).
- **La función hash genera una secuencia de bits.**
- En cualquier momento, solo se usa un sufijo/prefijo del binario para indexar los registros en una tabla de direcciones de buckets.
- **Crece duplicando directorio solo cuando es necesario.**

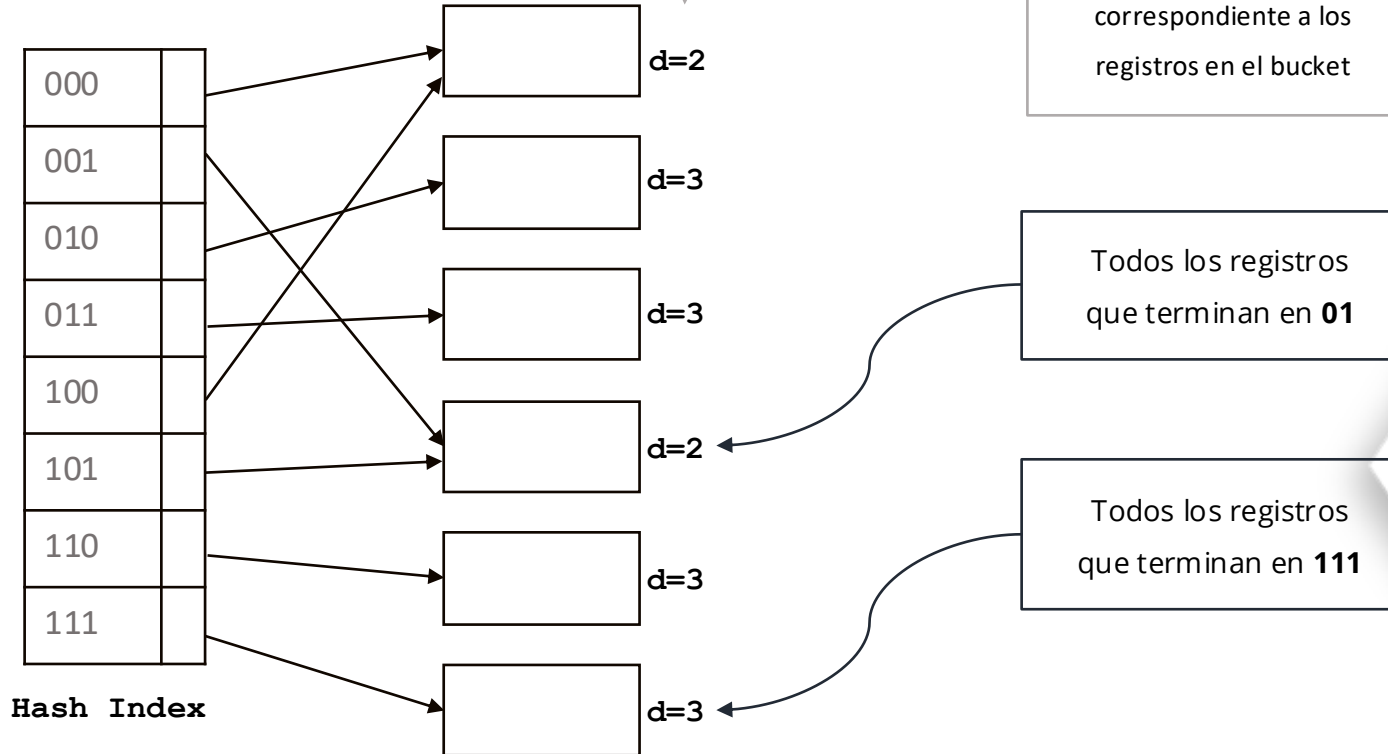


Extendible Hashing

<https://www.geeksforgeeks.org/dbms/extendible-hashing-dynamic-approach-to-dbms/>

Extendible Hashing

Profundidad
global $D = 3$



Extendible Hashing

Ejemplo

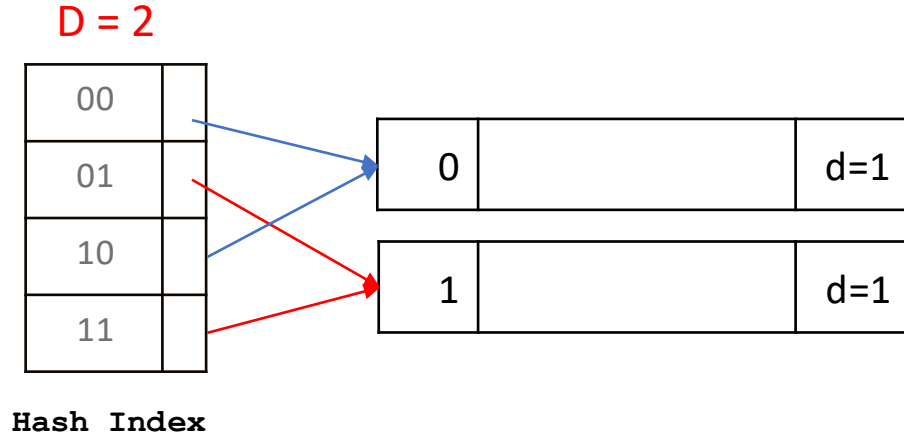
Key =

2,3,5,7,11,17,1

,8,19,23,28,29,

31,32,36

$\text{Bin}(\text{Key} \% 2^D)$



2 = 000010
3 = 000011
5 = 000101
7 = 000111
8 = 001000
11 = 001011
17 = 010001
18 = 010010
19 = 010011
23 = 010111
28 = 011100
29 = 011101
31 = 011111
32 = 100000
36 = 100100

Extendible Hashing

Ejemplo

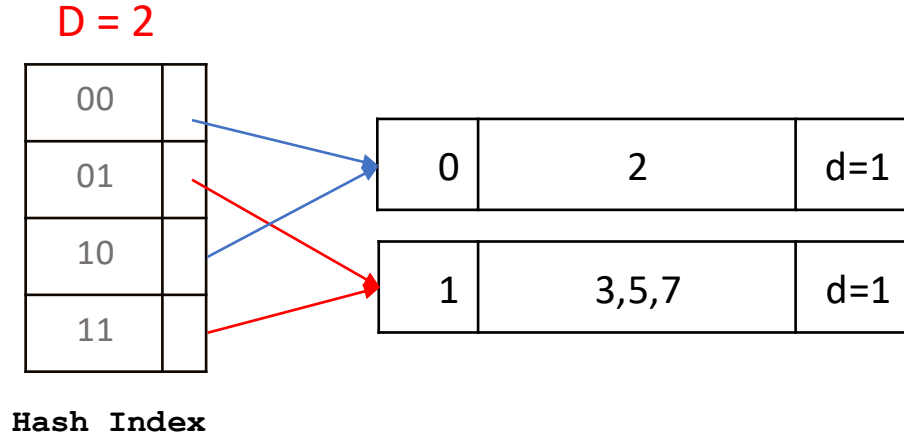
Key =

2,3,5,7,11,17,1

,8,19,23,28,29,

31,32,36

$\text{Bin}(\text{Key} \% 2^D)$



Factor de bloque: 3

Extendible Hashing

Ejemplo

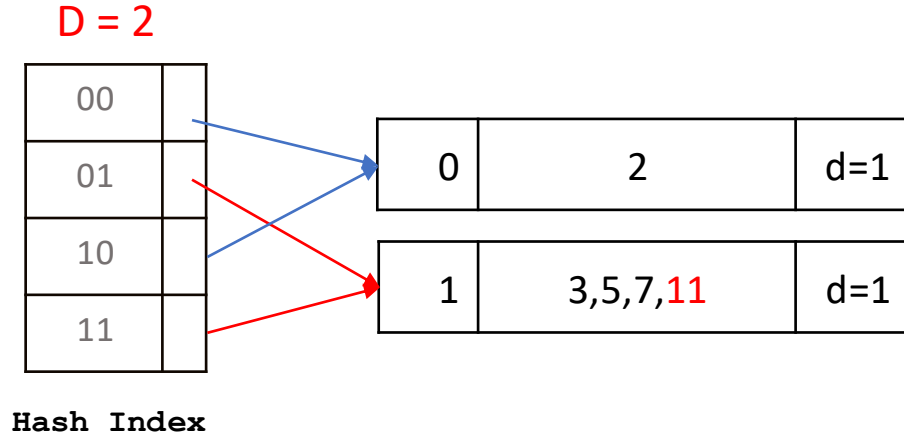
Key =

2,3,5,7,11,17,1

8,19,23,28,29,

31,32,36

$\text{Bin}(\text{Key} \% 2^D)$



Factor de bloque: 3

Extendible Hashing

Ejemplo

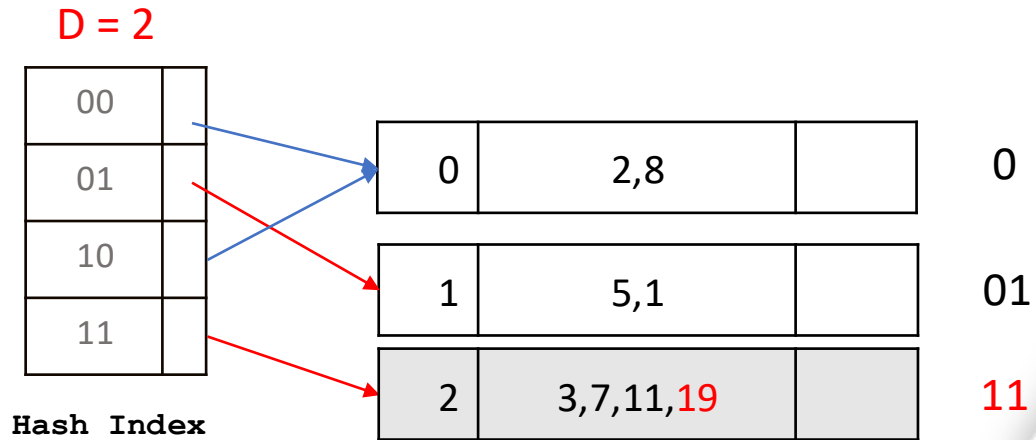
Key =

2,3,5,7,11,17,1

,8,19,23,28,29,

31,32,36

$\text{Bin}(\text{Key} \% 2^D)$



Extendible Hashing

Ejemplo

Key =

2,3,5,7,11,17,1

,8,19,23,28,29,

31,32,36

$\text{Bin}(\text{Key} \% 2^D)$

$D = 3$

000	
001	
010	
011	
100	
101	
110	
111	

Hash Index

Factor de bloque: 3

0	2,8, 28, 32	
1	5,1, 29	
2	3, 11,19	
3	7, 23, 31	

0

01

011

111

Extendible Hashing

Ejemplo

Key =

2,3,5,7,11,17,1

,8,19,23,28,29,

31,32,36, **27**

$\text{Bin}(\text{Key} \% 2^D)$

D = 3

000	
001	
010	
011	
100	
101	
110	
111	

Hash Index

Factor de bloque: 3

0	8, 32	
1	5, 1, 29	
2	3, 11, 19, 27	
3	7, 23, 31	
4	2	
5	28, 36	

000

01

011

111

10

100

Extendible Hashing

Factor de bloque: 3

$D = 3$

Ejemplo

Key =

2,3,5,7,11,17,1

,8,19,23,28,29,

31,32,36, **27**

$\text{Bin}(\text{Key} \% 2^D)$

000	
001	
010	
011	
100	
101	
110	
111	

Hash Index

0	8, 32	
1	5, 1, 29	
2	3, 11, 19	6
3	7, 23, 31	
4	2	
5	28, 36	
6	27	

000

01

011

111

10

100

Extendible Hashing

- **Búsqueda:**

- Sea **D** la profundidad global del índice.
- Aplicar la función hash sobre la clave de búsqueda y obtener la secuencia ***D-bit***.
- Haga coincidir la secuencia de ***D-bit*** con una entrada del directorio y dirigirse al bucket correspondiente para encontrar el registro.
 - El índice también provee información de la profundidad local de los buckets.

Extendible Hashing

- Inserción:

- Para insertar un registro se debe usar la secuencia ***D-bit*** para localizar el bucket respectiva.
- Si no encontró bucket, **proceder a buscar uno con la profundidad local mínima**. Si no lo encontró, proceder a crear el bucket.
- Si encontró un bucket y no esta lleno, proceder a insertar.
- Si el bucket esta lleno, dividir el bucket y reinsertar todos los registros.
 - Se crean nuevos buckets con una nueva profundidad local ($d = d + 1$).
 - El directorio es modificado.
- Si ya no se puede incrementar la profundidad, se produce desbordamiento de buckets.
- Si el desbordamiento es demasiado (parametrizar), entonces aumentar la profundidad global ($D = D + 1$)

Extendible Hashing

- **Eliminación:**

- Localizar el buffer respectivo y remover el registro.
- Si el bucket queda vacío, puede ser liberado.
 - Implica actualizar el índice.
- Si dos buckets tienen poco elementos y tienen el mismo prefijo en la profundidad local anterior ($d-1$), proceder a mezclar.
 - Implica actualizar el índice.

Extendible Hashing

- **Ventajas:**

- La eficiencia no se degrada con el crecimiento del archivo de datos.
- Minimiza el espacio de overflow.

- **Deficiencias:**

- El índice podría llegar a ser muy grande (mucho espacio en memoria principal).
- Cambiar el tamaño del índice (tabla de direcciones) es una operación cara.
- **No soporta búsqueda por rango.**

Otras técnicas de hashing dinámico

- Trie Hashing:
 - Usa un **Trie Digital** en lugar de un directorio plano.
 - Expande solo las ramas necesarias cuando hay colisiones.
 - Linear Hashing (LH):
 - Usa un **split secuencial** de buckets.
 - Mantiene un nivel n y un puntero de división p .
- ✓ *Fundamentals of Database Systems, Elmasri & Navathe (Sixth Edition), Chapter 18.*
✓ *Linear hashing: a new tool for file and table addressing. Litwin, W. (1980)*

Comparación de Algoritmos de Hashing Dinámico

Criterio	Extendible Hashing	Linear Hashing	Trie Hashing
Estructura Principal	Directorio en RAM (2^{GD}) + Buckets en disco	Sin directorio, Puntero Next (s) + Buckets en disco	Árbol digital de prefijos (Nodos internos + Hojas)
Mecanismo de División	División local; duplicación global si $LD = GD$	División secuencial round-robin por puntero Next	División por prefijo al desbordar hoja
Crecimiento	Exponencial a saltos (duplicación de RAM)	Lineal y progresivo (cubeta por cubeta)	Asimétrico y localizado por rama
Uso de RAM	Alto (almacena array de punteros)	Mínimo (sin directorio en RAM)	Eficiente (compresión de prefijos)
Casos de Uso	Búsquedas exactas (=) con suficiente RAM	RAM restringida y archivos dinámicos	Búsquedas por prefijo, expresiones regulares y autocompletado

2.



Índices Avanzandos

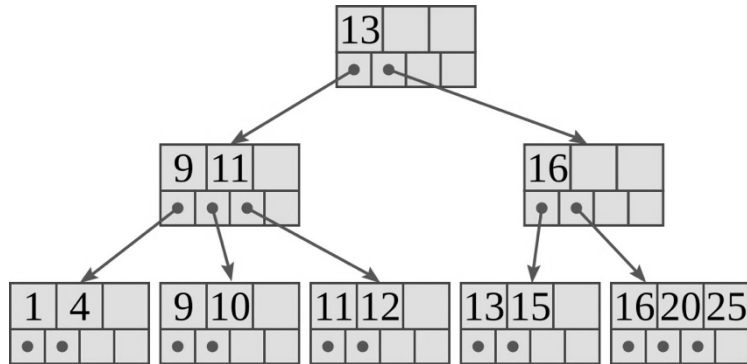
En PostgreSQL

1. BRIN
2. GiST
3. SP-GiST

[PostgreSQL: Documentation: Indexes](#)

Hellerstein et al. – Generalized Search Trees for Database Systems (GiST Paper)

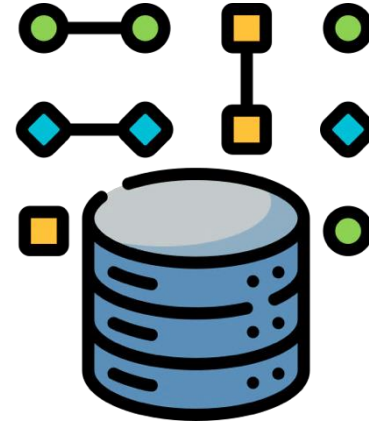
Limitaciones del B-Tree



- Óptimo para igualdad (=) y orden (<, >, BETWEEN).
- No es eficiente en búsquedas por similitud o intersección.
- No funciona bien con arrays, JSONB o datos espaciales.
- Índices B-Tree pueden ser grandes y costosos en inserciones masivas.

Necesidad de explorar índices avanzados

- Los datos no siempre se consultan de la misma forma.
- Necesidad de búsquedas más complejas (espacio, texto, jerarquías).
- Mejorar rendimiento con grandes volúmenes de datos.
- Reducir el tamaño de los índices (e.g. BRIN).
- Soporte para estructuras especializadas como arrays, documentos o geometrías.



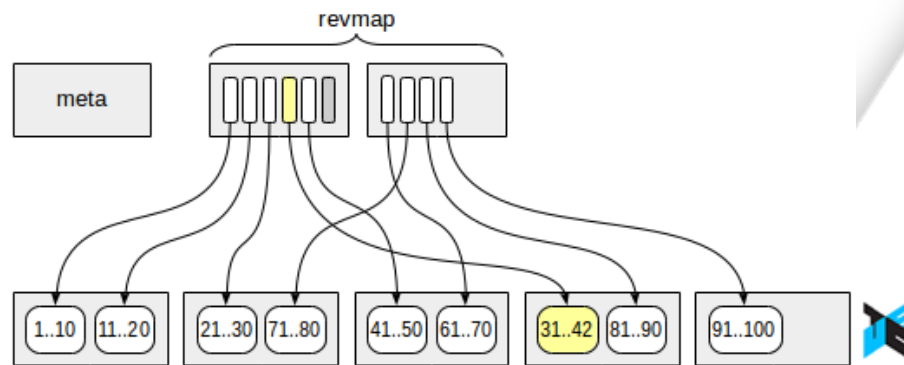
BRIN (Block Range Index)

Almacena resúmenes de bloques de datos (mínimo/máximo).

Ideal para tablas grandes con ordenamiento natural (e.g. fechas).

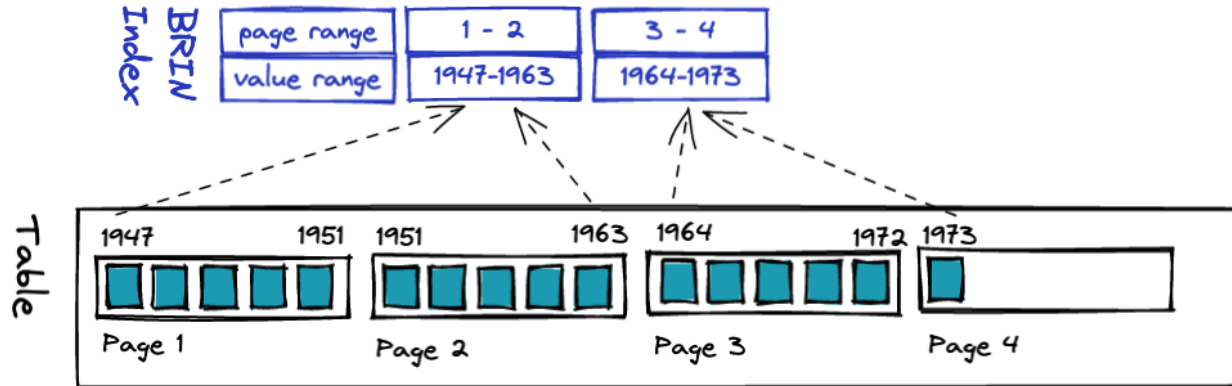
Índices muy compactos y rápidos de construir.

Consultas aproximadas que reducen el número de bloques leídos.



Cuanto más ordenados estén los datos físicamente, más eficiente será el BRIN.

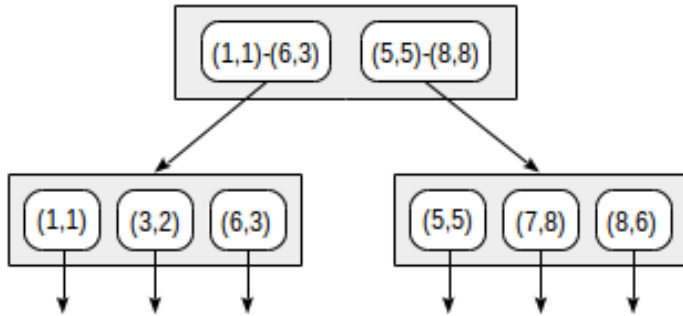
BRIN (Block Range Index)



```
CREATE INDEX idx_brin_sequential ON Table USING BRIN (year);
```

¿Que pasa si los registros no estan ordenados en la fecha?

GiST (Generalized Search Tree)



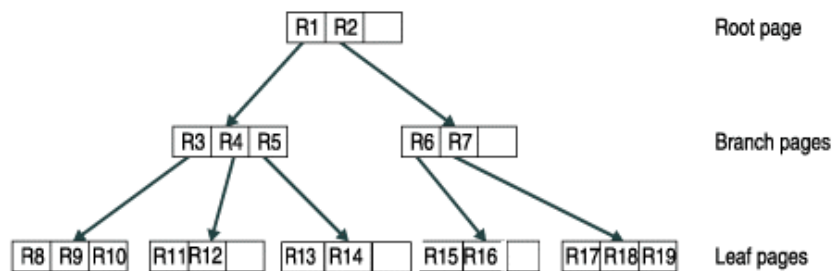
Soporta múltiples tipos de consultas: igualdad, rango, proximidad.

Índices para datos espaciales (PostGIS), texto completo, etc.

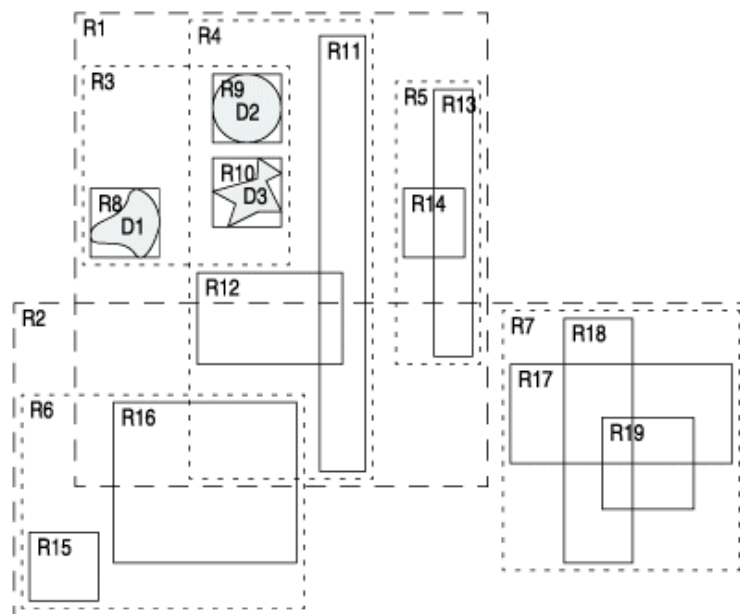
Estructura de árbol balanceado como B-Tree.

Flexible y extensible mediante operadores personalizados.

Soporta estructuras como R-Tree y Inverted Index.



GiST - RTree



SP-GiST (Space-Partitioned GiST)

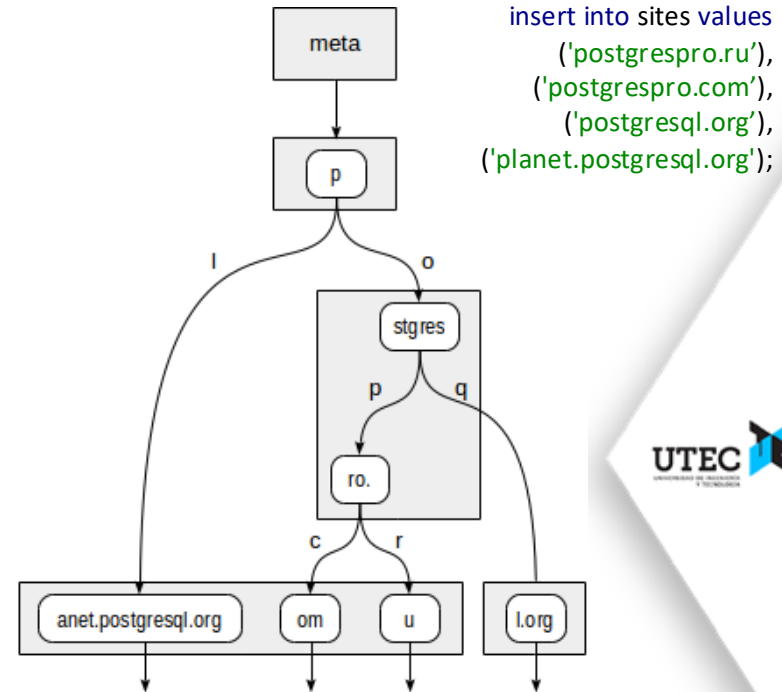
Índices basados en partición del espacio de datos.

Eficiente para estructuras como árboles trie, quadrees, k-d trees.

Útil en búsquedas espaciales y de prefijos.

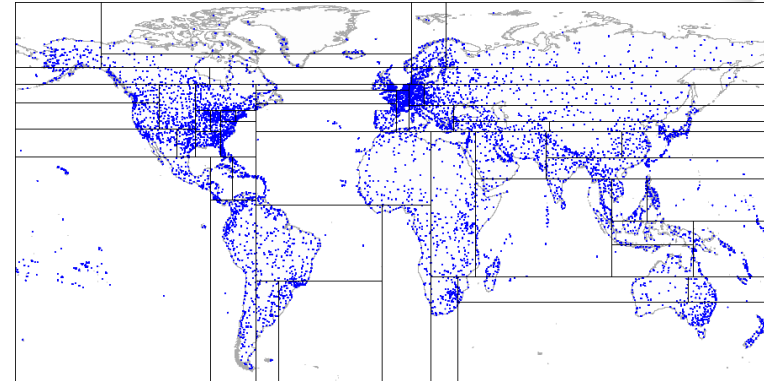
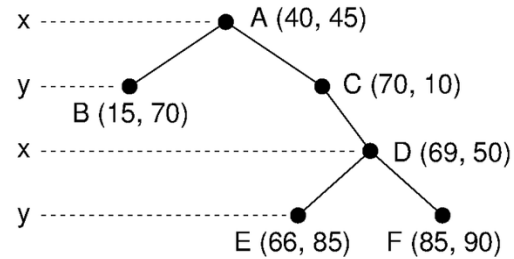
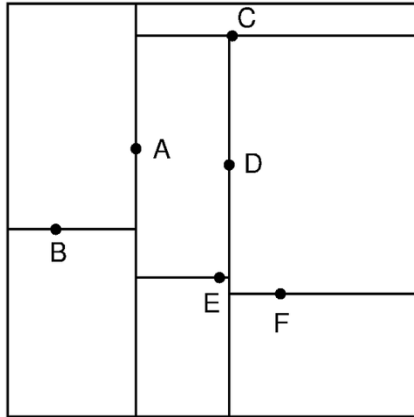
Permite crear índices especializados para ciertos tipos de datos.

No es adecuado para todos los tipos de consultas.



SP-GiST (Space-Partitioned GiST)

KD-Tree



Comparación General de Índices

B-Tree:

- Ideal para búsquedas por **igualdad** (=) y **orden** (<, >, BETWEEN).
- Es el **índice predeterminado**.

BRIN (Block Range INdexes).

- Eficiente para grandes volúmenes de datos que tienen un orden natural (por ejemplo, fechas o series de tiempo).
- Bajo costo de almacenamiento y mantenimiento.

GiST (Generalized Search Tree):

- Soporta consultas generales sobre datos espaciales, textuales y tipos personalizados.
- Ideal para búsquedas por proximidad, intersección, similitud, etc.

SP-GiST (Space-Partitioned GiST):

- Optimizado para **estructuras jerárquicas** (como árboles) y **espaciales** con partición natural (trie, quadrees, etc.).
Útil cuando los datos están naturalmente dispersos.

3.



Laboratorios 05

Resolver ejercicios